

Programming Languages Write-Up

Project Title

Car Rental Database and Management System

Course Connection

This project was created as a combined final project for CS3003 Programming Languages and CS4092 Database Design and Development.

For CS3003, the project demonstrates how Python can be used to build an application using object-oriented programming, imperative programming, functions, classes, control structures, and database interaction.

For CS4092, the project demonstrates how a relational database can be designed and implemented for a real-world system.

Programming Language Used

The main programming language used in this project is Python.

Python was chosen because it is readable, widely used, and works well for building command-line applications. Python also has built-in support for SQLite through the `sqlite3` library, which makes it useful for connecting application logic to a relational database.

Programming Paradigms Used

This project mainly uses two programming paradigms:

1. Object-oriented programming
2. Imperative programming

Object-Oriented Programming

Object-oriented programming is used in the `models.py` file. The project includes classes that represent real-world entities in the car rental system.

The main classes are:

- `Customer`
- `Staff`
- `Vehicle`
- `Rental`
- `Payment`
- `Maintenance`

Each class represents an important object in the system. For example, the **Vehicle** class represents a vehicle with attributes such as make, model, year, license plate, daily rate, and status.

Classes and Objects

The project uses classes to organize data and make the program easier to understand.

For example, the **Customer** class stores customer information such as first name, last name, phone number, email address, and driver's license number.

The **Vehicle** class stores vehicle information such as make, model, year, license plate, daily rate, and status.

These classes help connect the programming part of the project to the database design, because the classes match the main entities in the database.

Encapsulation

Encapsulation means grouping related data and behavior together inside a class.

In this project, each class groups related information into one object. For example, the **Vehicle** class keeps vehicle-related information together, and it also includes methods such as `get_vehicle_name()` and `is_available()`.

This makes the code more organized because vehicle data and vehicle-related behavior are kept in the same class.

Methods

The project includes methods inside classes.

Examples include:

- `get_full_name()` in the **Customer** class
- `get_full_name()` in the **Staff** class
- `get_vehicle_name()` in the **Vehicle** class
- `is_available()` in the **Vehicle** class

These methods help perform small tasks related to each object.

For example, `get_vehicle_name()` returns the year, make, and model of a vehicle in a readable format.

Imperative Programming

The project also uses imperative programming in the command-line interface.

Imperative programming means giving the computer step-by-step instructions to complete a task.

The `main.py` file uses this style. It shows a menu, asks the user for input, checks the user's choice, and then runs the correct function.

For example, when the user chooses to rent a vehicle, the program follows these steps:

1. Ask for customer ID, vehicle ID, and staff ID.
2. Ask for the start date and end date.
3. Check if the customer exists.
4. Check if the staff member exists.
5. Check if the vehicle exists.
6. Check if the vehicle is available.
7. Insert a rental record.
8. Update the vehicle status to rented.

This is an example of imperative programming because the program follows a clear sequence of instructions.

Control Structures

The project uses control structures such as:

- `if`
- `elif`
- `else`
- `while`
- `try`
- `except`

The `while` loop keeps the command-line menu running until the user chooses to exit.

The `if`, `elif`, and `else` statements decide which feature should run based on the user's menu choice.

The `try` and `except` blocks help handle invalid input or database errors.

Functions

The project uses functions to organize the program into smaller parts.

Some important functions are:

- `show_menu()`
- `view_available_vehicles()`
- `search_vehicles()`
- `add_customer()`
- `add_vehicle()`
- `rent_vehicle()`
- `return_vehicle()`

- `view_rental_history()`
- `view_payments()`
- `add_maintenance_record()`

Each function has one main purpose. This makes the program easier to read, test, and update.

Database Interaction

The project connects Python to a SQLite database using the `sqlite3` library.

The `database.py` file contains the database connection logic. The function `get_connection()` connects to the database, and the function `initialize_database()` runs the SQL script that creates the tables and inserts sample data.

The program sends SQL commands from Python to the database. These commands are used to insert, update, and retrieve data.

Examples of database interaction include:

- Viewing available vehicles
- Searching vehicles
- Adding a customer
- Adding a vehicle
- Creating a rental
- Returning a vehicle
- Viewing rental history
- Viewing payment records
- Adding maintenance records

Error Handling

The project uses basic error handling with `try` and `except`.

This helps prevent the program from crashing when the user enters invalid data or when the database returns an error.

For example, if the user enters text instead of a number for a vehicle ID, the program displays an error message instead of stopping unexpectedly.

Language Design Reflection

Python made this project easier because the syntax is simple and readable. The language allowed me to build the command-line menu, create functions, define classes, and connect to the database without too much extra setup.

Python also supports object-oriented programming while still allowing simple imperative programming. This made it a good language for this project because the system needed both organized classes and step-by-step database operations.

What I Learned

This project helped me understand how programming language concepts are used in a real application.

I learned how to organize code using classes and functions. I also learned how a Python program can connect to a database and run SQL commands. The project helped me see the connection between programming language design, application logic, and database design.

Conclusion

The Car Rental Database and Management System demonstrates how Python can be used to build a simple database-driven application.

The project uses object-oriented programming through model classes, imperative programming through the command-line menu, and database interaction through SQLite. It shows how programming language concepts can be applied to solve a real-world problem.